

- Adam optimizer with full batch and Gaussian noise added to the update direction for each parameter ($W \leftarrow W + \text{lr} \cdot (\Delta W + \epsilon)$, where ϵ is sampled from unit Gaussian, ΔW is the standard Adam weight update, and lr is the learning rate)
- Adam optimizer on model with residual dropout 0.1 added
- AdamW optimizer with weight decay 1 (default setting in most other experiments)
- AdamW optimizer with weight decay 1 towards the initialization instead of the origin
- Adam optimizer with learning rate $3 \cdot 10^{-4}$
- Adam optimizer with learning rate $3 \cdot 10^{-3}$
- Adam optimizer on model with Gaussian weight noise of standard deviation 0.01 (i.e. each parameter W replaced by $W + 0.01 \cdot \epsilon$ in the model, with ϵ sampled from unit Gaussian).

For experiments reported in Section 3.1.1 we increased the optimization budget to $5 \cdot 10^5$ optimization steps in order to capture the increase of time to perfect generalization better.

For the experiments reported in Section 3.1 we increased the optimization budget to 10^6 , and used Adam optimizer with no weight decay, for emphasizing how late into the optimization process the generalization can begin.

We’ve repeated each experiment for each dataset size with 3 random seeds, with the exception of experiments in section 3.1.1, where we’ve aggregated results over 7 random seeds.

A.2 ADDITIONAL FIGURES

In Figure 4 we show the loss curves that correspond to the accuracy curves in Figure 1.

In Figure 5 we show an example of a binary operation table that the network can actually solve.

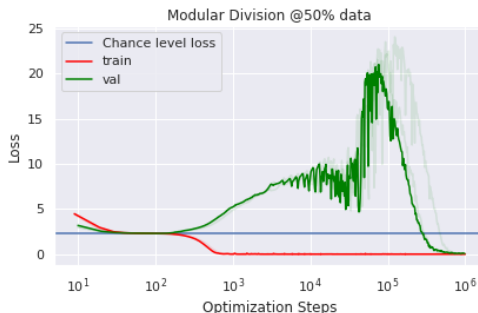


Figure 4: The loss curves for modular division, train and validation. We see the validation loss increases from 10^2 to about 10^5 optimization steps before it begins a second descent.

A.3 RELATED WORK

In this paper we study training and generalization dynamics on small simple algorithmic datasets. In the past, algorithmic datasets have been used to probe the capability of neural networks to perform symbolic and algorithmic reasoning. For example the tasks of copying, reversing, and sorting randomly generated sequences, and performing arithmetic operations of multi-digit numbers, have been used as standard benchmarks for sequence-to-sequence models Graves et al. (2014), Weston et al. (2014) Kaiser & Sutskever (2015) Reed & De Freitas (2015), Grefenstette et al. (2015), Zaremba & Sutskever (2015), Graves (2016), Dehghani et al. (2018). Typically in these works however the emphasis is on the performance in the unlimited data regime, with generalization often studied with respect to input sequence length. Some papers study the sample complexity on algorithmic tasks Reed & De Freitas (2015), but mostly focus on the impact of architectural choices. In contrast we study the phenomenon of generalization in data-limited regime, with an emphasis on phenomena that we believe to be architecture-agnostic.

Figure 5: One of the binary operation tables presented to the networks that the network can perfectly fill in. Each symbol is represented as a letter in English, Hebrew, or Greek alphabet for reader’s convenience. We invite the reader to guess which operation is represented here.

Algorithmically generated reasoning datasets like bAbI Weston et al. (2015) encourage work on studying generalization in data-limited regime. Most results on such datasets however focus on a point estimate of performance of a particular architecture or training technique, whereas our main interest is in pointing out the change in generalization past the point where a particular architecture can memorize the training data completely.

Neelakantan et al. (2015) has a “grok-like” learning curve on an algorithmic task, but it is related to optimization difficulty, whereas our phenomenon is specifically about generalization.

In Saxton et al. (2019) they study generalization on procedurally generated math problems such as arithmetic and differentiation, but for the most part these tasks are more involved than the simple binary op problems we have studied and as such do not lend themselves to observing the kinds of phenomena we describe in this paper, since they would require an extremely large number of samples to master.

In Jiang et al. (2019) they studied a large number of generalization or complexity measures on convolutional neural networks to see which, if any, are predictive of generalization performance. They find that flatness based measures that aim to quantify the sensitivity of the trained neural network to parameter perturbations are the most predictive. We conjectured that the grokking phenomena

we report in this work may be due to the noise from SGD driving the optimization to flatter/simpler solutions that generalize better and hope to investigate in future work whether any of these measures are predictive of grokking.

Zhang et al. (2016) finds that neural networks of sizes typically used in deep learning can interpolate arbitrary training data, and yet generalize when trained with semantically meaningful labels using appropriate optimization procedures. Our work shows a related phenomenon where neural networks can interpolate a small algorithmic training dataset without generalizing, but start generalizing when trained with SGD for longer.

Nakkiran et al. (2019); Belkin et al. (2018) focus on the phenomenon of double descent in loss as a function of model and optimization procedure capacity. They find that the classical U-shaped validation loss curve is followed in some settings (including neural network training) by a second descent of loss that starts around the minimal capacity that is needed to interpolate any training data. We observe a second descent in validation loss (though not accuracy) as a function of the amount of training in some of our experiments, and it happens past the point of interpolating the training data. We believe that the phenomenon we describe might be distinct from the double descent phenomena described in Nakkiran et al. (2019); Belkin et al. (2018) because we observe the second descent in loss far past the first time the training loss becomes very small (tens of thousands of epochs in some of our experiments), and we don't observe a non-monotonic behavior of accuracy. The setting of small algorithmic datasets that we study also provides a smaller, more tractable playground for studying subtle generalization phenomena than natural datasets studied in Nakkiran et al. (2019).

A.4 GENERALIZATION WITH MEMORIZING SEVERAL OUTLIERS

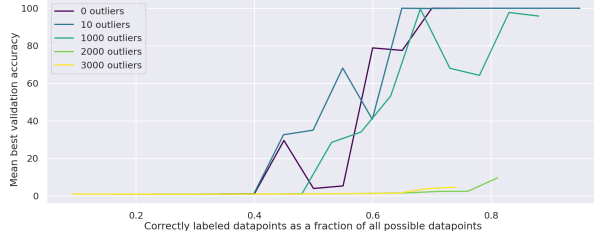


Figure 6: Effect on data efficiency of introducing $k \in [0, 10, 100, 1000, 2000, 3000]$ outliers (examples with random labels) into the training data. Small number of outliers doesn't noticeably impact generalization performance, but a large number hinders it significantly.

In this section we show data efficiency curves for a modified version of a binary op by introducing k outliers to the training dataset. More precisely, at the beginning of the experiment we randomly sample k equations from the training set and replace their answers with answers to other k equations randomly sampled from the training data. The rest of the equations in the training data and all the equations in the validation data are kept as before.

In this situation one could imagine one of the following scenarios unfolding. If the model class of neural networks optimized and regularized as before was not large enough to interpolate such "noisy" dataset, one could imagine the procedure converging to a solution that generalizes well, but denoises the training data (i.e. predicts $c = a \circ b$ as an answer even for the outlier equations $a, b \rightarrow c'$ with $c' \neq c$). On the other extreme it could be that the optimization procedure can find networks that interpolate the data, but the resulting models don't generalize, because they are forced to represent a considerably more complicated function than before (a simple function + k exceptions encoded in the training data).

In our experiments we find that the first option doesn't happen - all experiments reach 100% training accuracy at some point, and this point is not considerably affected by changing the number of outliers k . The second phenomenon happens in a range of training data percentages and number of outliers k - increasing k decreases the range of training data percentages for which the optimization procedure converges to models that generalize. However the effect of introducing a small number of outliers (up to 1000) is not very pronounced - see Figure 6. We interpret this as additional evidence that the capacity of the network and optimization procedure is well beyond the capacity needed for